

An FPGA Accelerator for Vision Transformer with Quantization and LUT-Based Operations*

Cheng XU^{†a)}, Yirong KAN^{†b)}, Renyuan ZHANG^{††c)}, *Nonmembers*, and Yasuhiko NAKASHIMA[†], *Fellow*

SUMMARY This paper proposes a Field-Programmable Gate Array (FPGA) accelerator for Vision Transformers (ViTs) with quantization and look-up-table (LUT) based operations. First, two improved quantization methods are proposed, achieving comparable performance at lower bit-widths. Furthermore, linear and nonlinear units' designs are proposed to support diverse operations in ViTs models. Finally, the LUT-based accelerator design is implemented and evaluated. Experimental results on the ImageNet dataset demonstrate that our proposed quantization method achieves an accuracy of 80.74% at 2-bit width, outperforming state-of-the-art Vision Transformer quantization methods by 0.1% to 0.5%. The performance of the proposed FPGA accelerator demonstrates a higher energy efficiency, achieving a peak energy efficiency of 7.06 FPS/W and 246 GOPS/W.

key words: vision transformer, FPGA, quantization, acceleration

1. Introduction

Over the past few years, the Transformer architecture has revolutionized the field of natural language processing (NLP). Inspired by its success in NLP, Transformer-based models, such as Vision Transformers (ViTs), have demonstrated remarkable capabilities across various computer vision (CV) tasks, often surpassing state-of-the-art convolutional neural networks (CNNs) in performance [1]–[3]. These tasks include image classification [4], [5], object detection [4], [6], and image super-resolution [7].

However, ViTs are characterized by their complex model structures, high computational demands, and significant memory requirements, often involving hundreds of millions of parameters. For instance, the ViT-L model contains approximately 307 million parameters, resulting in high memory consumption and extended inference times. Deploying such models on resource-constrained edge devices is a significant challenge. Additionally, many real-time CV applications demand low latency, making efficient deployment even more critical. Consequently, model compression

techniques tailored to Transformer-based architectures are essential for practical applications.

Model quantization has been extensively studied as a technique for reducing the computational complexity to accelerate DNN inference on edge devices. Existing quantization methods can be broadly categorized into post-training quantization (PTQ), quantization-aware training (QAT) and binary quantization.

PTQ methods apply quantization to pre-trained models without additional training, making them computationally efficient but often leading to accuracy degradation, especially for low-bit quantization. PTQ4ViT [8] proposed similarity-aware and ranking-aware quantization algorithms to find the optimal quantization interval of the MLP and self-attention layers. TSPTQ-ViT [9] proposed two-scaled post-training quantization scheme to reduce the quantization loss and improve accuracy.

In contrast, QAT methods [10], [11] incorporate quantization into the training process, allowing the model to adapt to quantization effect and achieve better accuracy. FPQ4ViT [10] proposed a learning-based Quantization-aware Training approach. Auto-ViT-Acc [11] proposed mixed-scheme quantization algorithm to improve computation efficiency and maintain accuracy.

Recent studies have explored low-bit quantization techniques for ViTs, aiming to improve efficiency while maintaining model accuracy. Binary and ternary quantization methods [12], [13] have been proposed to further reduce model size and computational complexity. Full-binarized ViTs [14] replace full-precision weights and activations with binary values, enabling efficient bitwise operations. However, these extreme quantization approaches often lead to significant accuracy degradation. Some studies [8], [11] have investigated mixed-precision quantization, allocating different bit-widths to different layers to balance efficiency and accuracy.

While PTQ methods offer a lightweight solution for quantization without requiring retraining, they often struggle with severe accuracy degradation under ultra-low-bit settings due to the limited flexibility in optimizing pre-trained weights. QAT methods improve accuracy by incorporating quantization into the training process, but they introduce additional computational overhead and require careful tuning to prevent unstable convergence. Full binary quantization further reduces model size and computational complexity but suffers from significant performance loss, making it impractical for ViT-based applications with high accuracy re-

Manuscript received January 9, 2025.

Manuscript revised April 22, 2025.

Manuscript publicized June 24, 2025.

[†]Division of Information Science, Nara Institute of Science and Technology, Ikoma-shi, 630-0192 Japan.

^{††}School of Information Science and Engineering, Yunnan University, Kunming, 650091, China.

*This paper was presented at the Twelfth International Symposium on Computing and Networking. This work was supported by JSPS KAKENHI Grant Numbers JP22H00515, JP22K21280, and JST-ALCA-Next Program Grant Number JPMJAN23F4, Japan.

a) E-mail: cheng.xu.db8@naist.ac.jp

b) E-mail: kan.yirong@is.naist.jp (Corresponding author)

c) E-mail: rzhang@is.naist.jp (Corresponding author)

DOI: 10.1587/transinf.2025PAP0003

quirements.

To address these challenges, this work proposes a hardware-aware quantization framework that builds upon QAT. Unlike conventional uniform quantization, our approach considers the weight distribution across different model layers that are often sparse and zero-centered and applies finer-granularity quantization to better preserve small-magnitude weights. This improves the trade-off between accuracy and hardware efficiency, enabling competitive performance under ultra-low-bit settings on FPGA platforms.

The contributions of our work are summarized as follows:

- Two quantization approaches are proposed to improve model inference accuracy and utilize FPGA LUT resource efficiently.
- A quantization-aware training algorithm is proposed to further enhance model performance.
- FPGA-specific algorithms and hardware structures are designed for linear and non-linear layers in Vision Transformers.
- An LUT-based vision transformer accelerator is implemented and optimized, offering significant improvements in accuracy and performance for resource-constrained edge devices.

This article extends the authors' previous work [15], which proposed FPGA-oriented quantization approaches for ViTs. This version brings updated related works (Sect. 1), explains detailed vision transformer literatures (Sect. 2), proposes design approaches for linear layers and nonlinear layers of vision transformer (Sect. 3), and provides additional evaluations with FPGA-based acceleration works (Sect. 4).

2. Preliminaries

This section provides an overview of the Vision Transformer (ViT) architecture and various quantization schemes. A detailed analysis of the ViT model components, particularly the implementation of its nonlinear parts, is presented, highlighting their importance in model inference. Furthermore, different quantization techniques, including uniform and non-uniform quantization intervals, are explored. By comparing and analyzing these techniques, we reveal their potential impact on resource utilization and computational performance in FPGA implementations.

2.1 Vision Transformer

The Transformer model, initially designed for NLP tasks, revolutionized the field by addressing the limitations of RNNs such as LSTMs and GRUs. Its encoder-decoder structure and self-attention mechanisms enable parallel computation, reducing training time and improving task performance, particularly in machine translation. As depicted in Fig. 1, the Vision Transformer (ViT) encoder comprises L identical layers, each containing two primary components: a multi-head

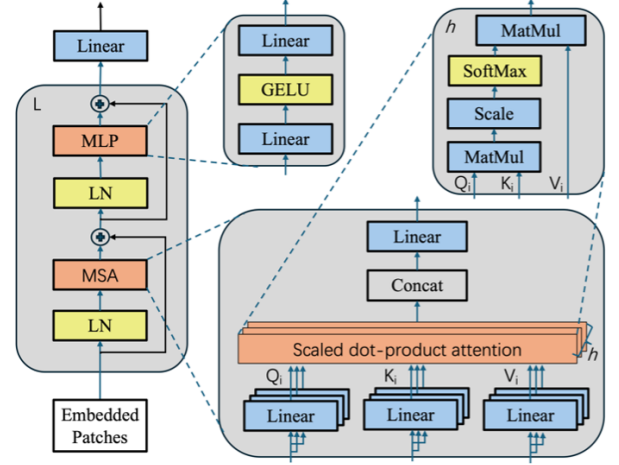


Fig. 1 Vision Transformer structure.

self-attention (MSA) mechanism and a multi-layer perceptron (MLP). Layer normalization (LN) [16] is applied prior to each sub-layer, and residual connections merge outputs with the input to preserve information flow.

ViT enhances feature extraction by employing multi-head self-attention, where multiple attention heads process data simultaneously, and their results are concatenated to produce the final output. The computational complexity of ViT arises primarily from extensive matrix multiplication operations in the MLP and MSA blocks, which dominate the overall workload. While quantization strategies typically target the linear layers to reduce computational demand, the nonlinear layers, such as softmax, layer normalization and GELU, which play a critical role in enhancing the model's expressive power, often remain in full precision due to their lower cost and critical impact on model accuracy.

The layer normalization function in Vision Transformer can be expressed as Eq. (1):

$$LN(x_{i,j}) = \alpha_j \times \frac{x_{i,j} - \mu_i}{\sqrt{\delta_i^2 + \epsilon}} + \beta_j \quad (1)$$

Where μ_i represents the mean of the inputs in the i -th row, δ_i^2 denotes the variance of the inputs in the i -th row, and ϵ is a very small constant added to avoid division by zero. Additionally, α_j and β_j represent the weight and bias for the j -th column, respectively.

To avoid overflow and potential precision loss, the Softmax function in Vision Transformers can be expressed as Eq. (2):

$$Softmax(x_{i,j}) = \frac{e^{x_{i,j} - x_{max_i}}}{\sum_{j=1}^n e^{x_{i,j} - x_{max_i}}} \quad (2)$$

Where $x_{i,j}$ represents the input data, and x_{max_i} denotes the maximum value among the n inputs along the column dimension.

The GELU function in Vision Transformers is approximated using Eq. (3):

$$GELU(x_{ij}) = \frac{x_{ij}}{2} \cdot \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} \right) (x_{ij} + 0.044715 x_{ij}^3) \right) \quad (3)$$

2.2 Quantization Schemes

Uniform interval quantization encompasses binary, ternary, and fixed-point approaches. Binary Neural Networks (BNNs) [12] and XNOR-Net [13] exemplify binary quantization, where restricting weights to $(-1, +1)$ allows replacing multiplications with additions/subtractions. Ternary quantization, by including zero, requires an additional bit for weight representation but similarly benefits from non-multiplicative operations. While both binary and ternary quantization significantly reduce computational complexity and simplify hardware accelerator implementations, they incur a noticeable accuracy penalty.

In contrast, fixed-point quantization employs moderate and adaptable bit-widths, enabling the maintenance of accuracy comparable to 32-bit floating-point models while improving efficiency.

For m -bit fixed-point quantization scheme [11], the quantized weights are defined as shown in Eq. (4):

$$Q^{FP}(m, \alpha) = \alpha \times \left\{ 0, \pm \frac{1}{2^{m-1}-1}, \pm \frac{2}{2^{m-1}-1}, \dots, \pm 1 \right\} \quad (4)$$

The definition of the quantizer is given in Eq. (5), which maps the 32-bit floating-point weights ω to the quantized weights represented by m -bit fixed-point values.

$$\hat{\omega} = \Pi_{Q^{FP}}(\omega) = \alpha \times \text{round}(h(\lceil \omega, \alpha \rceil)) \quad (5)$$

where $\Pi_{Q^{FP}}(\cdot)$ denotes the quantizer function to project onto $Q^{FP}(m, \alpha)$; the function $h(\cdot)$ transforms a value within $[-1, +1]$; and $\lceil \omega, \alpha \rceil$ clips ω by Eq. (6):

$$\lceil \omega, \alpha \rceil = \begin{cases} -1 & , \quad \omega < -\alpha \\ \omega/\alpha & , \quad -\alpha \leq \omega \leq \alpha \\ 1 & , \quad \omega > \alpha \end{cases} \quad (6)$$

To balance hardware efficiency and model accuracy, researchers have explored non-uniform quantization techniques. Among these, Power-of-Two (PoT) [17], [18] quantization significantly reduces computational complexity by substituting multiplication with bit shifts, greatly reducing complexity, but at the cost of precision—even with higher bit-widths.

The m -bit PoT quantized weight values are defined as shown in Eq. (7):

$$Q^{POT}(m, \alpha) = \alpha \times \left\{ 0, \pm \frac{1}{2^{2^{m-1}-2}}, \pm \frac{1}{2^{2^{m-1}-3}}, \dots, \pm 1 \right\} \quad (7)$$

Quantizing weights to the $2^b (b \in \mathbb{N})$ allows for the replacement of multiplication operations between weights and activations with highly efficient bit shifts, as shown in Eq. (8):

$$2^b \times a = \begin{cases} a < b & , \quad b > 0 \\ a & , \quad b = 0 \\ a > b & , \quad b < 0 \end{cases} \quad (8)$$

While power-of-two (PoT) quantization simplifies hardware implementation by avoiding multiplications, its precision gains are limited even with increasing quantization bit widths. Weights are commonly quantized using PoT, whereas activations typically employ fixed-point quantization. Applying PoT quantization to activations leads to a significant loss of precision.

3. Proposed Method

This section details the proposed quantization approach and design approach of accelerator.

3.1 Quantization Schemes

The proposed non-zero fixed-point (NFQ) quantization reduces computational cost and hardware area by eliminating zero values from the weight matrix, enabling shift-and-add operations instead of multipliers. This significantly lowers LUT usage, improves throughput, and better preserves small-magnitude weights, leading to higher model accuracy.

In the NFQ scheme, quantized weight values, represented using m bits, are defined as Eq. (9):

$$Q^{NFQ}(m, \alpha) = \alpha \times \left\{ \pm \frac{1}{2^{m-1}}, \pm \frac{2}{2^{m-1}}, \dots, \pm 1 \right\} \quad (9)$$

The mapping of a 32-bit floating-point weight ω to its m -bit NFQ quantized representation $\hat{\omega}$ is performed by the following quantizer Eq. (10):

$$\hat{\omega} = \Pi_{Q^{NFQ}}(\omega) = \alpha \times (h(\lceil \omega, \alpha \rceil)) \quad (10)$$

where $\Pi_{Q^{NFQ}}(\cdot)$ denotes the projection operator that maps a value onto the set $Q^{NFQ}(m, \alpha)$; the function $h(\cdot)$ is a clipping function that maps values into the range $[-1, +1]$; and $\lceil \omega, \alpha \rceil$ represents a clipping operation on ω defined as Eq. (11):

$$\lceil \omega, \alpha \rceil = \begin{cases} \text{Floor}(\omega/\alpha) & , \quad \omega < 0 \\ \text{Ceil}(1/\alpha) & , \quad \omega = 0 \\ \text{Ceil}(\omega/\alpha) & , \quad \omega > 0 \end{cases} \quad (11)$$

Figure 2 (b) illustrates the weight distribution of the first row in the QKV linear layer of a Vision Transformer model's first layer after NFQ quantization, contrasting it with the original weight distribution shown in Fig. 2 (a). By eliminating zero values from weight matrices, NFQ quantization provides an optimization tailored for FPGA hardware characteristics, leading to improved computational efficiency.

To optimize FPGA resource utilization, a scale power-of-2 (SP2) quantization scheme is proposed. While traditional power-of-2 quantization uses shifts for multiplication and addition, scaling factor multiplication still requires FPGA DSP resources. SP2 addresses this by also quantizing scaling factors to powers of 2, enabling shift operations for

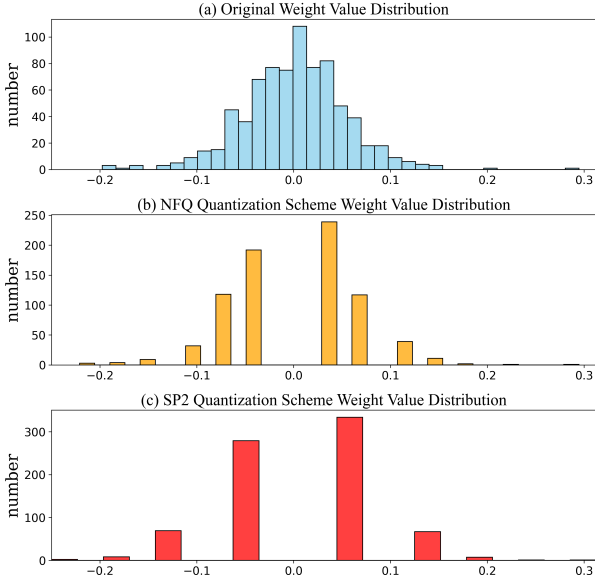


Fig. 2 Weight value distribution.

all multiplications and improving LUT efficiency. Formally, given m -bit representations, the SP2 quantized weight values are Eq. (12):

$$Q^{SP2}(m, \alpha) = \alpha \times \left\{ \pm \frac{1}{2^{m-1}}, \pm \frac{2}{2^{m-1}}, \dots, \pm 1 \right\} \quad (12)$$

Where n -bit scaling factor α representation is Eq. (13):

$$\alpha = \{2^{-n}, 2^{-n+1}, \dots, \pm 1\} \quad (13)$$

And the mapping from a 32-bit floating-point weight ω into the quantized weight $\hat{\omega}$ by m -bit representation in SP2 scheme is given by the following quantizer Eq. (14):

$$\hat{\omega} = \Pi_{Q^{SP2}} \omega = 2^{c(\lceil \omega, n \rceil)} \times h(\lceil \omega, \alpha \rceil) \quad (14)$$

where $\Pi_{Q^{SP2}}(\cdot)$ denotes the quantizer function to project on $Q^{SP2}(m, \alpha)$; the function $h(\cdot)$ mapping value into the range of $[-1, +1]$; and function $c(\cdot)$ mapping max weight value into the value of power-of-2.

Figure 2(c) illustrates the weight distribution of the first row within the first-layer QKV linear layer of the Vision Transformer when employing SP2 quantization. This distribution exhibits greater value-level sparsity, with more weights concentrated on fewer quantized values, compared to those observed in the original model Fig. 2(a) and in the NFQ scheme Fig. 2(b).

As indicated by Eq. (6), zero values in the weight matrix are converted to the smallest positive value. However, this conversion can be implemented using either positive or negative values. To evaluate the effects of these two approaches, experiments were conducted. The accuracy loss curves for different zero value handling methods under the NFQ quantization schemes after 50 training epochs. As shown in the Fig. 3, the accuracy loss curves for both positive and negative zero value handling are very close over the 50 training

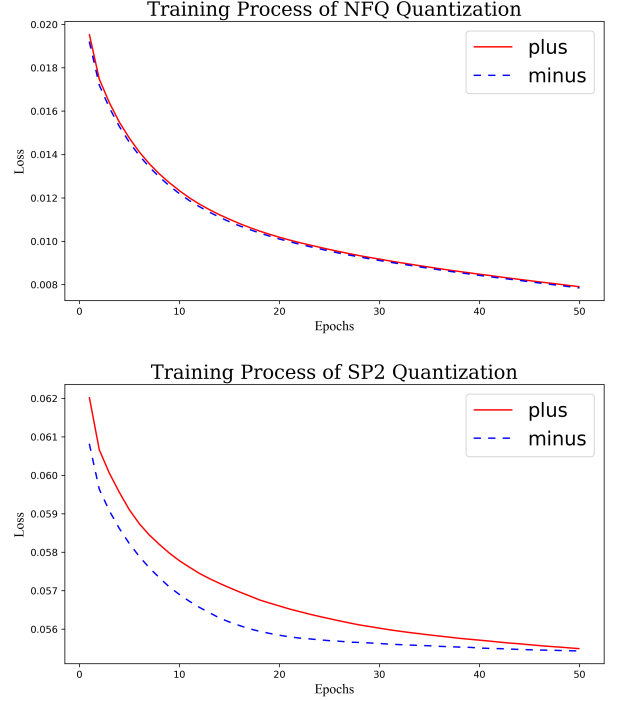


Fig. 3 The loss curve in training processes.

Algorithm 1 FPGA-aware ViT Quantization

Input: 32-bit floating-point pre-trained ViT model $\mathcal{M}$ with weights $\mathcal{W}$ to be quantized.

Quantization scheme: $\mathbb{S} \in \text{Non-zero symmetry, Scale Power-of-2}$

target: Quantized model $\hat{\mathcal{M}}$

```

foreach epoch do
  foreach batch do
    // forward propagation
    foreach layer  $i$  in  $\mathcal{M}$  do
      foreach row  $M_{ij}$  in layer  $i$  do
         $\hat{W}_{ij} \leftarrow \Pi_{\mathbb{S}}^{\mathbb{S}}(W_{ij})$ 
    // backward propagation
    loss  $\leftarrow \mathcal{M}(\hat{\mathcal{W}})$ 
    foreach layer  $i$  (reverse order) do
       $\frac{\partial \text{Loss}}{\partial W_{ij}} \leftarrow \frac{\partial \text{Loss}}{\partial \hat{W}_{ij}}$ 
      update  $\mathcal{W}$ 
  foreach epoch do
    foreach batch  $b$  in input do
      // forward propagation
      foreach row  $j$  in batch  $b$  do
        // quantize activations
         $\hat{A}_{b,j} = \Pi_{\mathbb{S}}^{\mathbb{S}}(A_{b,j})$ 
        // backward propagation
        loss  $\leftarrow \mathcal{M}(\hat{A})$ 
        backpropagate loss and update  $\mathcal{W}$ 
return  $\hat{\mathcal{M}} \leftarrow \mathcal{M}(\hat{\mathcal{W}})$ 

```

epochs. The accuracy loss curves for positive and negative value handling are nearly identical, with an actual inference Top-1 and Top-5 accuracy difference of approximately 0.1%, which is negligible.

Algorithm 1 outlines our proposed FPGA-oriented Vision Transformer quantization training procedure, which op-

Algorithm 2 Resource search algorithm

Input: ViT model M
target: space search results $(N_i, N_o), Throughput_{num}, Cycles_{num}$
Initialize (N_i, N_o)
for n_i **in** $stepList$ **do**
 for n_o **in** $stepList$ **do**
 if $n_i \times n_o \leq maxValue$ **then**
 $(N_i, N_o).append(n_i, n_o)$

//Iterate through each layer of the model
foreach (n_i, n_o) **in** (N_i, N_o) **do**
 Initialize $macs_{total}, cycles_{total}$
 foreach $layer$ **in** M **do**
 $macs_{layer} = calculate_mac(layer)$
 $cycles_{layer} = calculate_cycles(layer)$
 $macs_{total} += macs_{layer}$
 $cycles_{total} += cycles_{layer}$
 $throughput = macs_{total} / cycles_{total}$
 $Throughput_{num}.append(throughput)$
 $Cycles_{num}.append(Cycles_{total})$
return $(N_i, N_o), Throughput_{num}, Cycles_{num}$

erates with defined bit-widths for weights and activations. In this work, we investigate two weight quantization schemes, NFQ and SP2, and apply NFQ for activation quantization.

3.2 Accelerator Design

In FPGA design, consolidating multiple smaller-scale designs into a single large-scale design can significantly reduce complexity while improving energy efficiency. The FPGA computing array is typically implemented as a systolic array comprising multiple computing units, parameterized by using (n_i, n_o) , where n_i signifies the input channels, and n_o represents the output channels. To optimize the computing array configuration, an exhaustive search Algorithm 2 is proposed to aim at maximizing effective throughput. This method evaluated the actual throughput across different (n_i, n_o) configurations to identify the optimal solution. Specifically, all possible (n_i, n_o) combinations are iterated for all layers to calculate the total throughput. Under the quantization scheme adopted in this work, the configurations are set as follows: W2A6 – (54, 128), W4A8 – (64, 64), and W8A8 – (64, 32).

The proposed FPGA accelerator for Transformers is illustrated in Fig. 4, and the structure of the Processing Element is shown in Fig. 5. To enable both attention layers and linear layer computations on a single large-scale computing array, the design incorporates three types of processing elements (PEs): PE0, PE1 and PE2. Each PE supports two operational modes, corresponding to the computation requirements of attention layers and linear layers, respectively. During execution, preprocessed partitioned blocks are transferred from DRAM to either the input buffer or the weight buffer. The computing array then retrieves data and weights from these buffers for computation. The output buffer feeds the data into the nonlinear acceleration unit. Once processed, the data is either written back to DRAM or transferred to the input buffer or weight buffer for subsequent computations.

The GELU layer in Vision Transformers, as described in

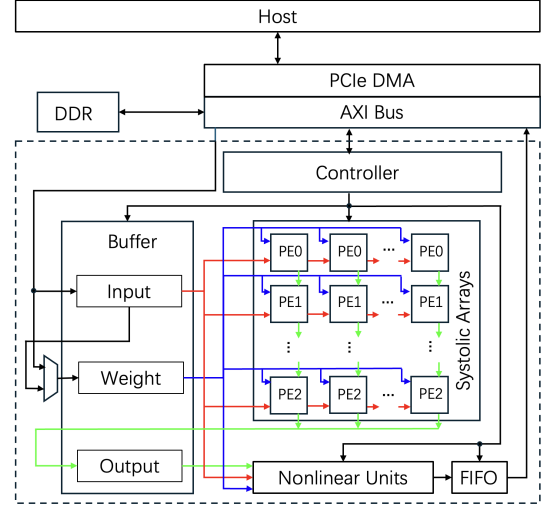


Fig. 4 Overview of the proposed accelerator architecture.

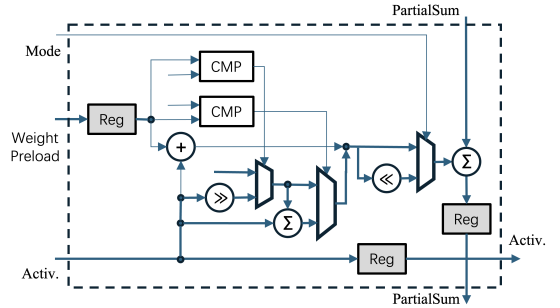


Fig. 5 PE architecture.

Eq. (3), can be further approximated for simpler computation as Eq. (15):

$$GELU(x_{ij}) \approx \frac{x_{ij}}{2} \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}} x_{ij}\right) \right) = \frac{x_{ij}}{1 + e^{-\sqrt{\frac{8}{\pi}} x_{ij}}} \quad (15)$$

The softmax layer in Vision Transformers, as described in Eq. (2), includes the exponential term e^x . Direct hardware implementation of this exponential function on FPGAs is prohibitively expensive in terms of hardware resources. Consequently, an approximation of e^x is employed to reduce the computational burden.

$$e^x = \left(1 \ll \left\lfloor \frac{x}{\ln 2} \right\rfloor \right) + \left(1 + \{x\} + \frac{\{x\}^2}{2} + \dots \right) \quad (16)$$

Since directly applying the Taylor expansion to e^x results in low precision for the integer part, the computation is divided into two separate components, as illustrated in Eq. (16). The hardware structure for the Exp function with a Taylor expansion with three terms is shown in Fig. 6.

Both the GELU and softmax layers involve reciprocal calculations. The reciprocal is computed using the iterative formula (Eq. (17)), and the hardware structure for the reciprocal calculation is shown in Fig. 7.

$$\frac{1}{x} : x^{n+1} = 2x^n - x_n^2 \cdot x \quad (17)$$

In Eq. (17), the initial value of x_n is determined by the initial value of x .

The layer normalization function in Vision Transformers, as described in Eq. (1), involves the Reciprocal square root calculation. The reciprocal square root function is computed using the iterative formula (Eq. (18)), and the hardware structure for the reciprocal square root function is shown in Fig. 8.

$$\frac{1}{\sqrt{x}} : x_{n+1} = x_n \cdot \frac{3 - x \cdot x_n^2}{2} \quad (18)$$

Similarly, In Eq. (18) the initial value of x_n is dependent

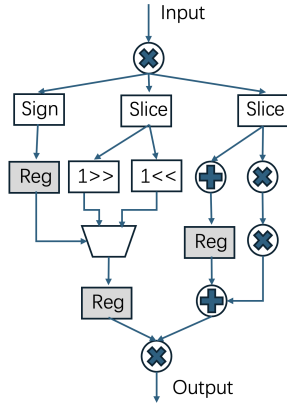


Fig. 6 Hardware architecture of Exp function.

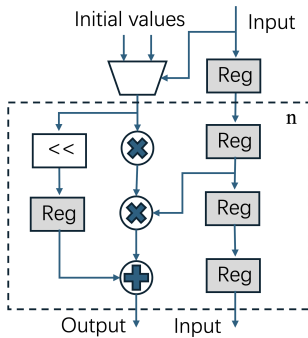


Fig. 7 Hardware architecture of Reciprocal function.

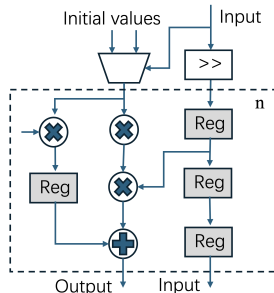


Fig. 8 Hardware architecture of Reciprocal Square Root function.

on the initial value of x .

4. Experiments

This section details the experimental setup, including configurations and parameters. The proposed Vision Transformer quantization schemes are evaluated on ImageNet, with performance compared against a baseline and alternative methods, including AVA [11], MSQ [19], and PTQ [8]. The proposed FPGA acceleration is similarly compared to the baseline and AVA [11].

4.1 Experimental Setups

Experiment investigates the quantization implementation of a Vision Transformer model based on DeiT-base, excluding token sparsification. Quantization-aware training (QAT) with varying bit-widths was performed using a batch size of 300, preceded by a 300-epoch pre-training phase. The AdamW optimizer (weight decay: 0.05, initial learning rate: 1×10^{-4}) was employed, with learning rate decay governed by the ReduceLROnPlateau scheduler (factor: 0.1, patience: 3). All quantized models trained with identical hyperparameters on Ubuntu systems with CUDA 12.1 and PyTorch 2.3.0, using RTX 3090 and RTX 4090 GPUs, were then evaluated on a Xilinx ZCU102 FPGA platform. This platform features 2520 DSPs and 274.1k LUTs, and all designs, implemented using Xilinx Vitis and Vitis HLS 2020.2, operated at a frequency of 150 MHz. The official DeiT model (W32A32) served as the baseline.

4.2 Experimental Results

Table 1 compares the accuracy achieved by various quantization schemes applied to Vision Transformer. Activations use uniform NFQ quantization, while weights follow the methods listed in the first column. As shown, compared to prior methods, NFQ enables lower bit-width quantization with

Table 1 Accuracy Results under different Quantization Schemes

Quantization Scheme	Bit-Width Weight/Acti.	Model Accuracy(%)	
		Top-1	Top-5
ImageNet			
Deit-base	W32A32	81.85	95.59
MSQ	W4A4	70.27(-11.58)	89.42(-6.17)
MSQ	W4A32	71.31(-10.54)	90.11(-5.48)
PTQ	W6A6	77.02(-4.83)	-
PTQ	W8A8	80.48(-1.37)	-
AVA Fixed	W4A4	81.33(-0.52)	95.63(+0.04)
AVA Fixed	W4A8	81.93(+0.08)	95.90(+0.31)
AVA PoT	W3A4	80.87(-0.98)	95.97(+0.38)
AVA PoT	W4A8	81.51(-0.34)	95.73(+0.14)
NFQ(our)	W2A6	80.74(-1.11)	95.29(-0.30)
NFQ	W2A8	81.55(-0.30)	95.77(+0.18)
NFQ	W4A4	81.41(-0.44)	96.05(+0.46)
NFQ	W4A8	81.95(+0.10)	96.02(+0.43)
SP2(our)	W2A6	76.59(-5.26)	91.98(-3.61)
SP2	W2A8	77.32(-4.52)	92.55(-3.04)
SP2	W4A4	74.45(-7.40)	92.42(-3.17)
SP2	W4A8	81.61(-0.24)	95.47(-0.02)

Table 2 Resource Utilization under different Schemes

Scheme	Bit-Width Weight/Acti.	Resource Utilization			
		DSP	kLUT	kRegister	bram
DeiT-base					
Baseline	W32A32	1564	120	-	-
Fixed	W8A8	2066	128	-	-
Fixed	W4A4	2064	139	-	-
PoT	W4A8	20	192	-	-
PoT	W3A4	19	191	-	-
NFQ(our)	W2A6	0	163	145	262
NFQ	W4A8	0	160	96	319
NFQ	W8A8	0	152	63	508
SP2(our)	W2A6	0	160	145	262
SP2	W4A8	0	157	95	319
SP2	W8A8	0	148	63	508

Table 3 Performance under different Schemes

Scheme		Power (W)	Thrpt. (GOPS)	FR. (FPS)	EnergyEff. (FPS/W)
DeiT-base					
Baseline	W32A32	9.91	345.8	10.0	1.01
Fixed	W8A8	9.40	899.6	25.9	2.76
Fixed	W4A4	11.27	1648.1	47.5	4.21
PoT	W4A8	7.24	1080.5	31.1	4.30
PoT	W3A4	8.11	1958.4	56.4	6.95
NFQ(our)	W2A6	6.54	1597.2	45.8	7.00
NFQ	W4A8	5.89	881.8	25.2	4.28
NFQ	W8A8	5.10	515.6	14.7	2.88
SP2(our)	W2A6	6.49	1597.2	45.8	7.06
SP2	W4A8	5.85	881.8	25.2	4.33
SP2	W8A8	5.06	515.6	14.7	2.92

minimal accuracy degradation. Furthermore, at equivalent bit-widths, NFQ offers superior accuracy compared to other schemes. SP2 also maintains acceptable accuracy.

Using the NFQ quantization scheme, the following results were obtained: With 2-bit weights and 6-bit activations, Top-1 and Top-5 accuracies reached 80.74% and 95.29%, respectively, approximately 1% lower than the baseline. Increasing the activation bit-width to 8 (while maintaining 2-bit weights) yielded Top-1 and Top-5 accuracies of 81.55% and 95.77%. With 4-bit weights and activations, Top-1 accuracy was 81.41% and Top-5 accuracy was 96.25%, again less than 1% below the baseline. Notably, with 4-bit weights and 8-bit activations, both Top-1 (82%) and Top-5 (96%) accuracies surpassed the baseline by nearly 1%.

With the SP2 quantization scheme, using 2-bit weights and 6-bit activations yielded Top-1 and Top-5 accuracies of 76.59% and 91.98%, respectively (approximately 5% lower than the baseline Top-1 accuracy). Increasing the activation bit-width to 8 (with 2-bit weights) resulted in Top-1 and Top-5 accuracies of 77.32% and 92.55%. With 4-bit weights and activations, the Top-1 and Top-5 accuracies were 74.45% and 92.42%. Notably, using 4-bit weights and 8-bit activations achieved a Top-1 accuracy of 81.61% and a Top-5 accuracy of 95.47%, closely approaching the baseline (within 1%).

Table 2 compares the resource Utilization achieved by various quantization schemes of Vision Transformer. The two proposed quantization schemes differ in scaling factor handling: NFQ uses conventional scalars, while SP2 restricts

them to powers of 2. SP2 reduces LUT usage by a few thousand units and register usage by a few hundred units due to simpler arithmetic. Both schemes have identical BRAM consumption, as memory footprint remains unaffected by the scaling factor.

Table 3 compares the performance achieved by various quantization schemes of ViT. SP2 achieves slightly better energy efficiency and a marginally higher frame rate (<0.1 fps), making it suitable for resource or energy constrained applications. Compared to the Fixed and PoT schemes, Both NFQ and SP2 schemes exhibit a slightly lower frame rate but demonstrates significantly lower energy consumption, resulting in a higher energy efficiency ratio.

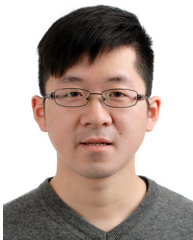
Conclusions

This paper investigates FPGA acceleration for Vision Transformer with quantization and LUT-based operations. Two quantization schemes and quantization-aware training algorithm are proposed to enhance model performance. The FPGA-specific algorithms and hardware structures are designed for linear and non-linear layers of vision transformers. Finally, a comprehensive comparison of the quantization algorithms and LUT-based accelerator implementations was conducted, evaluating accuracy, resource utilization, and performance.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L.u. Kaiser, and I. Polosukhin, "Attention is all you need," Advances in Neural Information Processing Systems, I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol.30. Curran Associates, Inc., 2017.
- [2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," ArXiv, vol.abs/2010.11929, 2020.
- [3] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. J'egou, "Training data-efficient image transformers & distillation through attention," in International Conference on Machine Learning, 2020.
- [4] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp.9992–10002, 2021.
- [5] O.N. Manzari, H. Ahmadabadi, H. Kashiani, S.B. Shokouhi, and A. Ayatollahi, "Medvit: A robust vision transformer for generalized medical image classification," Computers in biology and medicine, vol.157, p.106791, 2023.
- [6] C. Zhang, J. Su, Y. Ju, K.M. Lam, and Q. Wang, "Efficient inductive vision transformer for oriented object detection in remote sensing imagery," IEEE Trans. Geosci. Remote Sens., vol.61, pp.1–20, 2023.
- [7] C. Wang, T.J. Lu, G. Wu, Y. Wang, and Z. Sun, "D-esrgan: A dual-encoder gan with residual cnn and vision transformer for iris image super-resolution," 2022 IEEE International Joint Conference on Biometrics (IJCB), pp.1–8, 2022.
- [8] Z. Liu, Y. Wang, K. Han, S. Ma, and W. Gao, "Post-training quantization for vision transformer," Neural Information Processing Systems, 2021.
- [9] Y.-S. Tai, M.-G. Lin, and A.-Y.A. Wu, "TSPTQ-ViT: Two-Scaled

- Post-Training Quantization for Vision Transformer,” 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp.1–5, 2023
- [10] Z. Li, P. Wang, Z. Wang, and J. Cheng, “Fixed-point Quantization for Vision Transformer,” 2021 China Automation Congress (CAC), pp.7282–7287, 2021
- [11] Z. Li, M. Sun, A. Lu, H. Ma, G. Yuan, Y. Xie, H. Tang, Y. Li, M.E. Leeser, Z. Wang, X. Lin, and Z. Fang, “Auto-vit-acc: An fpga-aware automatic acceleration framework for vision transformer with mixed-scheme quantization,” 2022 32nd International Conference on Field-Programmable Logic and Applications (FPL), pp.109–116, 2022.
- [12] C. Lazarus and M.J. Kochenderfer, “A mixed integer programming approach for verifying properties of binarized neural networks,” ArXiv, vol.abs/2203.07078, 2022.
- [13] M. Mohaimenuzzaman, C. Bergmeir, and B. Meyer, “Pruning vs xnor-net: A comprehensive study of deep learning for audio classification on edge-devices,” IEEE Access, vol.10, pp.6696–6707, 2022.
- [14] Y. Li, S. Xu, M. Lin, X. Cao, C. Liu, X. Sun, and B. Zhang, “Bi-ViT: Pushing the Limit of Vision Transformer Quantization,” AAAI Conference on Artificial Intelligence, vol.38, no.4, pp.3243–3251, 2024.
- [15] C. Xu, Y. Kan, R. Zhang, and Y. Nakashima, “An FPGA-Oriented Quantization Approach for Vision Transformer with LUT-Friendly Operations,” 2024 Twelfth International Symposium on Computing and Networking (CANDAR), Naha, Japan, 2024, pp.266–271. doi: 10.1109/CANDAR64496.2024.00042.
- [16] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On layer normalization in the transformer architecture,” ArXiv, vol.abs/2002.04745, 2020.
- [17] P. Li, J. Cao, F. Li, H. Yao, and Y. Wang, “Trainable power-of-2 scale factors for hardware-friendly network quantization,” 2022 2nd International Conference on Computer, Control and Robotics (ICCCR), pp.151–155, 2022.
- [18] H.T. Tesfai, H.H. Saleh, M. Al-Qutayri, B. Mohammad, and T. Stouraitis, “Gradient estimation for ultra low precision pot and additive pot quantization,” IEEE Access, vol.11, pp.61264–61272, 2023.
- [19] S.-E. Chang, Y. Li, M. Sun, R. Shi, H. K.-H. So, X. Qian, Y. Wang, and X. Lin, “Mix and match: A novel fpga-centric deep neural network quantization framework,” 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp.208–220, 2021.



Cheng Xu received the B.S. degree in optical information science and technology and M.S. degree in software engineering from Huazhong University of Science and Technology, Wuhan, China, in 1999 and 2003, respectively. He was a Lecturer with the School of Computer Science, Yangtze University, Jingzhou, China. He is currently a Ph.D. student at the Architecture Lab in Nara Institute of Science and Technology, Japan. His research interests include deep learning, computer network, computer architecture.



and emerging computing technologies.

Yirong Kan received the B.E. degree from the Chengdu University of Technology, Chengdu, China, in 2015, the M.E. degree from Yunnan University, Kunming, China, in 2019, and the Ph.D. degree from the Nara Institute of Science and Technology, Ikoma, Japan, in 2022. He has been an Assistant Professor with the Division of Information Science, Nara Institute of Science and Technology since 2022. His current research interests include reconfigurable computing, computer architecture, digital IC design,



nan University as a Professor and NAIST as an Affiliated Professor. His research interests include analog–digital-mixed circuits, approximate computing, high performance computing, and hardware implementation of AI.

Renyuan Zhang received the BE degree from Tongji University, in 2007, the ME degree from Waseda University, in 2010, and the PhD degree from the University of Tokyo, in 2013. He was an assistant professor with Japan Advanced Institute of Science and Technology from 2013 to 2017. He has been an Assistant Professor with Nara Institute of Science and Technology (NAIST) and the PRESTO researcher with Japan Science and Technology Agency since 2017 and 2018, respectively. From 2024, he serves Yun-



ogy. His research interests include computer architecture, emulation, circuit design, and accelerators. He is a fellow of IEICE, a Senior Member of IPSJ, and a member of the IEEE CS and ACM.

Yasuhiko Nakashima received the B.E., M.E., and Ph.D. degrees in computer engineering from Kyoto University, in 1986, 1988, and 1998, respectively. He was a Computer Architect with the Computer and System Architecture Department, Fujitsu Ltd., from 1988 to 1999. From 1999 to 2005, he was an Associate Professor with the Graduate School of Economics, Kyoto University. Since 2006, he has been a Professor with the Graduate School of Information Science, Nara Institute of Science and Technol-